Vulnerability Triage Report

Generated by vulntriage · 2026-07-01 22:50

Executive Summary

This report covers 22 vulnerability finding(s) triaged by the LLM-driven pipeline. Of these, 2 are rated High exploitability, 7 Medium, and 13 Low. The highest-priority finding is: Sensitive Configuration Files Listing - Detect (severity: medium) [tags: config, listing, exposure, edb, vuln] (host 172.19.0.2, risk score 0.665). Findings are ranked by a composite risk score combining CVSS, LLM-assessed exploitability, and asset criticality. Recommended remediation steps follow each finding.

22	2	7	13
TOTAL FINDINGS	HIGH	MEDIUM	LOW

Risk Breakdown

#1 Sensitive Configuration Files Listing - Detect (…
0.665

#2 DVWA Default Login (severity: critical) [tags: d…
0.65

#3 Cookies without Secure attribute - Detect (sever…
0.5

#4 Cookies without HttpOnly attribute - Detect (sev…
0.5

#5 FingerprintHub Technology Fingerprint (severity:…
0.5

#6 Missing Cookie SameSite Strict (severity: info) …
0.5

#7 README.md file disclosure (severity: info) [tags…
0.5

#8 HTTP Missing Security Headers (severity: info) […
0.5

#9 HTTP Missing Security Headers (severity: info) […
0.5

#10 WAF Detection (severity: info) [tags: waf, tech,…
0.38

#11 Gitignore Config - Detect (severity: info) [tags…

0.38

#12 Wappalyzer Technology Detection (severity: info)…

0.38

#13 HTTP Missing Security Headers (severity: info) […

0.38

#14 HTTP Missing Security Headers (severity: info) […

0.38

#15 HTTP Missing Security Headers (severity: info) […

0.38

#16 HTTP Missing Security Headers (severity: info) […

0.38

#17 HTTP Missing Security Headers (severity: info) […

0.38

#18 HTTP Missing Security Headers (severity: info) […

0.38

#19 HTTP Missing Security Headers (severity: info) […

0.38

#20 HTTP Missing Security Headers (severity: info) […

0.38

#21 robots.txt endpoint prober (severity: info) [tag…

0.38

#22 robots.txt file (severity: info) [tags: miscella…

0.38

Technical Findings

#1 High Sensitive Configuration Files Listing - Detect (severity: medium) [tags: config, listing, exposure, edb, vuln]

Host: 172.19.0.2

CVSS: 5.3

Risk score: 0.665 **(asset criticality 0.5)**

Context: The vulnerability allows an attacker to list sensitive configuration files (e.g., .env, config.php, or database credentials) on the host 172.19.0.2, likely due to improper access controls or directory listing enabled. In a real-world attack, an adversary could exploit this to retrieve hardcoded secrets or API keys, then pivot to internal systems or escalate privileges. The business impact includes potential data breaches, compliance violations (e.g., GDPR, PCI-DSS), and reputational damage from exposed credentials. With a CVSS score of 5.3 (medium), immediate remediation is advised to prevent lateral movement or data exfiltration.

Exploitability rationale: The vulnerability allows listing of sensitive configuration files without authentication, and public exploits exist for similar misconfigurations, enabling easy credential theft and lateral movement.

Remediation: Disable directory listing and restrict access to sensitive configuration files to prevent unauthorized disclosure.

1. Identify the web server software (e.g., Apache, Nginx, IIS) and disable directory listing globally or for specific directories.
2. Move sensitive configuration files (e.g., .env, config.php, database.yml) outside the web root directory.
3. Implement access controls using server-specific directives (e.g., Apache .htaccess, Nginx location blocks) to deny access to known sensitive file patterns.
4. Review and remove any unnecessary files from the web-accessible directory that may expose configuration details.
5. Verify the fix by attempting to list directories and access sensitive files from an external browser or tool.

#2 High DVWA Default Login (severity: critical) [tags: dvwa, default-login, vuln]

Host: 172.19.0.2

Risk score: 0.65 **(asset criticality 0.5)**

Context: The DVWA web application on host 172.19.0.2 uses default login credentials (e.g., admin:password), which are widely known and easily discoverable. An attacker can log in with these credentials to gain unauthorized access, then exploit other intentionally built vulnerabilities like SQL injection or command injection, potentially leading to remote code execution or data exfiltration. In a production scenario, this could result in a full system compromise, sensitive data leakage, and significant reputational or regulatory damage.

Exploitability rationale: Default credentials are publicly known and easily exploitable, allowing immediate unauthorized access and subsequent exploitation of other vulnerabilities.

Remediation: Immediately disable the default admin credentials and reconfigure DVWA with strong, unique passwords to prevent unauthorized access.

1. Log into DVWA with the default credentials (e.g., admin/password) and change the admin password to a strong, complex password.
2. Edit the config/config.inc.php file to set the password field to the new secure password and update any other default settings (e.g., set 'security_level' to 'impossible').
3. Remove or disable any default user accounts that are not needed, and enforce account lockout policies if supported.

#3 Medium Cookies without Secure attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]

Host: 172.19.0.2

Risk score: 0.5 **(asset criticality 0.5)**

Context: The vulnerability is that cookies issued by the web application on host 172.19.0.2 lack the Secure attribute, allowing them to be transmitted over unencrypted HTTP connections. In a real-world attack, an adversary on the same network (e.g., public Wi-Fi) could perform a man-in-the-middle attack to intercept these cookies, leading to session hijacking and unauthorized access to user accounts. The business impact includes potential data breaches, loss of customer trust, regulatory fines (e.g., under GDPR or PCI DSS), and disruption of services due to compromised sessions.

Exploitability rationale: Exploitation requires network access to the internal network and man-in-the-middle capabilities, but no authentication bypass or public exploit is needed.

Remediation: Set the Secure flag on all cookies to ensure they are only transmitted over HTTPS, preventing exposure over unencrypted connections.

1. Review the web application configuration and code to identify all cookies set without the Secure attribute.
2. Modify the application or server configuration to include the Secure flag for each cookie, e.g., in PHP use session_set_cookie_params(['secure' => true]) or setcookie() with the secure parameter.
3. Ensure that HTTPS is enforced site-wide to support the Secure flag; consider implementing HTTP Strict Transport Security (HSTS) to force HTTPS.
4. Test the fix by using browser developer tools or a tool like curl to verify that cookies now have the Secure attribute set when served over HTTPS.

#4 Medium Cookies without HttpOnly attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]

Host: 172.19.0.2

Risk score: 0.5 **(asset criticality 0.5)**

Context: Cookies without the HttpOnly attribute are vulnerable to client-side script access, typically via cross-site scripting (XSS) attacks. An attacker exploiting an XSS vulnerability can steal session cookies, impersonate the victim, and perform unauthorized actions. This can lead to account takeover, sensitive data exposure, and reputational damage for the organization.

Exploitability rationale: The missing HttpOnly attribute increases risk of session theft via XSS, but exploitation requires an additional XSS vulnerability and user interaction, and no public exploit or CVE is available.

Remediation: Set the HttpOnly flag on all cookies to prevent client-side script access and mitigate XSS-based cookie theft.

1. Identify all cookies set by the application on 172.19.0.2 that lack the HttpOnly attribute.
2. Modify the application code to include the HttpOnly flag when setting cookies (e.g., in PHP: setcookie() with httponly=true; in Java: cookie.setHttpOnly(true); in ASP.NET: httpOnlyCookies=true in web.config).
3. For web servers like Apache or Nginx, use header-modifying directives (e.g., Header edit Set-Cookie ^(.*)$ $1;HttpOnly in Apache) to add the flag if application code changes are not immediately feasible.
4. Test the changes by inspecting HTTP response headers to confirm all cookies now include the HttpOnly attribute.
5. Deploy the updated configuration or code to production after successful testing.

#5 Medium FingerprintHub Technology Fingerprint (severity: info) [tags: tech, discovery]

Host: 172.19.0.2

Risk score: 0.5 **(asset criticality 0.5)**

Context: The target host 172.19.0.2 has a technology fingerprint identified by FingerprintHub, which reveals specific software or frameworks in use. While this finding itself is not a vulnerability, it provides attackers with critical reconnaissance data to tailor attacks against known weaknesses in those technologies. An attacker could cross-reference the fingerprint with vulnerability databases to exploit unpatched versions, leading to system compromise or data breach. The business impact includes increased risk of targeted attacks, potential loss of sensitive data, and reputational damage if the host supports critical operations.

Exploitability rationale: The fingerprint provides reconnaissance data that could be used to target unpatched versions, but the lack of a specific vulnerability or CVE and unknown service exposure reduces immediate exploitability.

Remediation: Reduce information disclosure by minimizing exposed technology fingerprints that could aid attackers in targeting specific software versions.

1. Review the identified technologies on host 172.19.0.2 and determine if each service is necessary for business operations.
2. Disable or remove any unnecessary services or applications that are not required.
3. Configure services to use generic banners or disable version disclosure in server headers and application responses.
4. Ensure all identified technologies are updated to the latest stable versions to mitigate known vulnerabilities.

#6 Medium Missing Cookie SameSite Strict (severity: info) [tags: misconfig, samesite, cookie, vuln]

Host: 172.19.0.2

Risk score: 0.5 **(asset criticality 0.5)**

Context: This finding indicates that cookies set by the web application on host 172.19.0.2 lack the SameSite=Strict attribute, making them vulnerable to Cross-Site Request Forgery (CSRF) attacks. An attacker could lure a user to a malicious page that triggers unwanted actions (e.g., password change, fund transfer) using the user's authenticated session. In real-world scenarios, such misconfigurations have led to account takeovers and data breaches. The business impact includes unauthorized operations, reputational damage, and potential financial loss, particularly if the application handles sensitive transactions.

Exploitability rationale: The missing SameSite attribute enables CSRF attacks, but exploitation requires user interaction and the service is on an internal IP, reducing immediate exposure.

Remediation: Set the SameSite attribute to Strict on all cookies to prevent CSRF attacks and limit cross-site request forgery.

1. Identify all cookies set by the application on 172.19.0.2.
2. For each cookie, add the SameSite=Strict attribute in the Set-Cookie header.
3. If using a web framework (e.g., Express, Django, Spring), configure the framework to enforce SameSite=Strict globally.
4. Test the application to ensure cookies are sent only for same-site requests and functionality is not broken.
5. Deploy the changes and verify via browser developer tools or a security scanner that the SameSite attribute is present.

#7 Medium README.md file disclosure (severity: info) [tags: exposure, markdown, files, vuln]

Host: 172.19.0.2

Risk score: 0.5 **(asset criticality 0.5)**

Context: The vulnerability involves a README.md file being accessible without authentication, exposing project information. An attacker could discover this file via directory enumeration or search engine dorking, potentially extracting sensitive data such as credentials, API keys, or internal documentation. In a real-world scenario, this could lead to unauthorized access if the file contains hardcoded passwords or configuration details. The business impact includes information leakage, compliance risks, and potential reputational damage if sensitive information is exposed.

Exploitability rationale: The README.md file is accessible without authentication on an internal host, and while no public exploits exist, low attack complexity and potential exposure of sensitive data could enable further attacks.

Remediation: Restrict access to documentation files to prevent information leakage by configuring the web server to deny access to .md files or moving them outside the web root.

1. Identify the web server software (e.g., Apache, Nginx, IIS) running on the host.
2. Configure the web server to block access to all .md files (e.g., in Apache: <FilesMatch "\.md$"> Require all denied </FilesMatch>; in Nginx: location ~ \.md$ { deny all; }).
3. Move or remove any unnecessary documentation files (like README.md) from the publicly accessible web root directory.
4. Review the web root for other sensitive or metadata files (e.g., .txt, .pdf, .yml) and apply similar access controls or remove them.
5. Test the remediation by attempting to access the README.md file (e.g., curl http://172.19.0.2/README.md) and confirm it returns 403 Forbidden or 404 Not Found.

#8 Medium HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.5 **(asset criticality 0.5)**

Context: The web server at 172.19.0.2 is missing critical HTTP security headers, such as Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options, which are essential for mitigating client-side attacks. In a real-world scenario, an attacker could exploit the missing X-Frame-Options to perform clickjacking, tricking users into performing unintended actions on the vulnerable site, or leverage the absence of Content-Security-Policy to inject malicious scripts via cross-site scripting (XSS). The business impact includes potential data breaches, session hijacking, and reputational damage from successful attacks that compromise user trust and regulatory compliance.

Exploitability rationale: Missing security headers increase attack surface for client-side attacks like clickjacking and XSS, but exploitation requires user interaction and no public exploit is directly available.

Remediation: Add the missing security headers to the web server configuration to mitigate common web vulnerabilities such as clickjacking, MIME sniffing, and XSS.

1. Identify the specific security headers that are missing by reviewing the HTTP response headers of the target application.
2. For Apache, add the following directives to the virtual host configuration or .htaccess file: Header always set X-Content-Type-Options nosniff; Header always set X-Frame-Options SAMEORIGIN; Header always set X-XSS-Protection "1; mode=block"; Header always set Referrer-Policy strict-origin-when-cross-origin; Header always set Content-Security-Policy "default-src 'self'"; Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains" (if HTTPS is used).
3. For Nginx, add the following lines to the server block: add_header X-Content-Type-Options nosniff; add_header X-Frame-Options SAMEORIGIN; add_header X-XSS-Protection "1; mode=block"; add_header Referrer-Policy strict-origin-when-cross-origin; add_header Content-Security-Policy "default-src 'self'"; add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" (if HTTPS).
4. For IIS, add the headers via the web.config file under <system.webServer><httpProtocol><customHeaders>: <add name="X-Content-Type-Options" value="nosniff" />, <add name="X-Frame-Options" value="SAMEORIGIN" />, etc.
5. Test the changes by sending a request to the server and verifying the new headers appear in the response.
6. Deploy the configuration changes to production after successful testing, and monitor for any application functionality issues.

#9 Medium HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.5 **(asset criticality 0.5)**

Context: The web server at 172.19.0.2 is missing critical HTTP security headers like Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security. This misconfiguration can be exploited in real-world attacks such as clickjacking, cross-site scripting (XSS), and man-in-the-middle attacks, allowing attackers to hijack user sessions, steal sensitive data, or inject malicious content. The business impact includes potential data breaches, loss of customer trust, regulatory penalties, and reputational harm, especially if the server handles sensitive user information.

Exploitability rationale: Missing security headers increase attack surface for client-side attacks, but exploitation requires user interaction or additional vulnerabilities, and no public exploit or CVE is available.

Remediation: Add missing HTTP security headers to harden the web application against common attacks like XSS, clickjacking, and MIME-type sniffing.

1. Identify the web server software (e.g., Apache, Nginx, IIS) and locate its configuration files.
2. Add the 'Content-Security-Policy' header to restrict sources of content, e.g., default-src 'self'.
3. Add the 'X-Content-Type-Options' header set to 'nosniff' to prevent MIME-type sniffing.
4. Add the 'X-Frame-Options' header set to 'DENY' or 'SAMEORIGIN' to prevent clickjacking.
5. Add the 'Strict-Transport-Security' header if HTTPS is used, with a suitable max-age and includeSubDomains.
6. Add the 'Referrer-Policy' header to control referrer information, e.g., 'no-referrer-when-downgrade'.
7. Add the 'Permissions-Policy' header to restrict browser features, e.g., 'camera=(), microphone=()'.
8. Add the 'X-XSS-Protection' header (though deprecated, still widely used) set to '1; mode=block'.
9. Test the headers using browser developer tools or an online checker like securityheaders.com.
10. Restart or reload the web server to apply changes and verify they are served in responses.

#10 Low WAF Detection (severity: info) [tags: waf, tech, misc, discovery]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: This finding indicates that a Web Application Firewall (WAF) has been detected on the host 172.19.0.2. While a WAF is a security control, its detection can aid attackers in fingerprinting the environment and tailoring bypass techniques. In a real-world scenario, an attacker could use this information to identify WAF rules and craft payloads that evade detection, potentially leading to successful web application attacks such as SQL injection or cross-site scripting. The business impact is moderate, as the presence of a WAF suggests an attempt to protect web assets, but its detection may reduce the security advantage it provides, increasing the risk of exploitation.

Exploitability rationale: The finding is informational (WAF detection) with no associated vulnerability or exploit, requiring prior web application attack capability for any potential bypass.

Remediation: WAF detection is informational and not a vulnerability; if obfuscation is desired, implement measures to reduce fingerprinting.

1. Evaluate whether hiding the WAF presence is necessary for your security posture.
2. If needed, configure the WAF to strip or modify identifiable headers and cookies from responses.
3. Implement custom error pages that do not reveal WAF vendor or version information.
4. Test the obfuscation changes to ensure they do not break legitimate traffic.
5. Document the WAF presence and detection status for future security assessments.

#11 Low Gitignore Config - Detect (severity: info) [tags: exposure, tenable, config, git, vuln]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The exposed .gitignore configuration file on host 172.19.0.2 provides attackers with a blueprint of the project's file structure and reveals intentionally ignored files, such as configuration files, logs, or sensitive data paths. In a real-world scenario, an attacker could leverage this information to target specific files for theft or to map the application's architecture, facilitating further attacks. Business impact includes increased risk of targeted attacks and data breaches, as the exposure aids reconnaissance and undermines defense-in-depth, even though the finding is low severity.

Exploitability rationale: The exposed .gitignore file provides reconnaissance information but does not directly enable exploitation, requires additional attack steps, and is likely on an internal network.

Remediation: Remove or secure the .gitignore file to prevent exposure of sensitive configuration patterns.

1. Verify the presence and contents of the .gitignore file on the host.
2. If the file is not needed, delete it using 'rm .gitignore'.
3. If the file is required, move it outside the web-accessible directory or restrict access via web server configuration (e.g., .htaccess or Nginx deny rules).
4. Review and remove any sensitive patterns (e.g., passwords, API keys) from the file.
5. Ensure .gitignore is not tracked in version control if it contains sensitive data; add it to .gitignore itself if necessary.

#12 Low Wappalyzer Technology Detection (severity: info) [tags: tech, discovery]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The Wappalyzer technology detection finding on host 172.19.0.2 does not identify a specific vulnerability but reveals the software stack in use, such as web servers, frameworks, or CMS platforms. In a real-world attack, an adversary could leverage this information to tailor exploits against known vulnerabilities in the identified technologies—for example, targeting an outdated version of Apache or a vulnerable plugin in WordPress. The business impact includes increased risk of targeted attacks, potential data breaches, and reputational damage if sensitive systems are compromised due to unpatched software.

Exploitability rationale: This is an informational finding with no specific vulnerability, CVE, or CVSS score, and the exploitability depends on the attacker's ability to find and exploit other vulnerabilities in the detected technologies, which is not directly assessed here.

Remediation: Reduce information disclosure by removing or obfuscating identifiable technology signatures in HTTP responses.

1. Review HTTP response headers (e.g., Server, X-Powered-By, X-AspNet-Version) for technology indicators and remove or set them to generic values.
2. Customize or disable default error pages and directory listings that may reveal software versions.
3. Use a reverse proxy or web application firewall (WAF) to strip or modify revealing headers before they reach the client.
4. Minimize usage of query parameters or URL patterns that disclose backend technologies (e.g., .php, .aspx).
5. Ensure that no detailed version information is included in HTML comments, JavaScript files, or CSS files.

#13 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The web server at 172.19.0.2 is missing common HTTP security headers, such as Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options. This misconfiguration can be exploited in real-world attacks like clickjacking (by embedding the site in an iframe), cross-site scripting (if CSP is absent), or MIME-type sniffing leading to drive-by downloads. Although classified as informational severity, the absence of these headers increases the attack surface and could lead to data theft, session hijacking, or reputational damage if exploited.

Exploitability rationale: Missing security headers are a misconfiguration that increases attack surface but require additional vulnerabilities or user interaction for exploitation, and the service is internal.

Remediation: Add recommended HTTP security headers to the web server configuration to mitigate common client-side attacks.

1. Identify the web server software (e.g., Apache, Nginx, IIS) and locate its configuration file.
2. Add the 'X-Content-Type-Options: nosniff' header to prevent MIME type sniffing.
3. Add the 'X-Frame-Options: SAMEORIGIN' or 'DENY' header to prevent clickjacking.
4. Add the 'X-XSS-Protection: 1; mode=block' header to enable browser XSS filters (note: deprecated but still commonly used).
5. Add a 'Content-Security-Policy' header to control resource loading and reduce XSS risks.
6. If HTTPS is used, add 'Strict-Transport-Security: max-age=31536000; includeSubDomains' header.
7. Add a 'Referrer-Policy' header (e.g., 'strict-origin-when-cross-origin') to control referrer information.
8. Restart the web server to apply changes.
9. Verify the new headers are present using curl -I or browser developer tools.

#14 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The web server at 172.19.0.2 is missing common HTTP security headers, which are critical for enforcing browser-based protections against attacks such as cross-site scripting (XSS), clickjacking, and MIME type sniffing. In a real-world scenario, an attacker could exploit the absence of Content-Security-Policy to inject malicious scripts, or lack of X-Frame-Options to perform clickjacking, leading to credential theft or unauthorized actions. The business impact includes potential data breaches, reputational harm from compromised user sessions, and non-compliance with security standards like OWASP Top 10 or industry regulations.

Exploitability rationale: Missing HTTP security headers are a common misconfiguration that typically requires additional vulnerabilities or user interaction to exploit, and no public exploit or CVE is associated with this finding.

Remediation: Add recommended HTTP security headers to the web server configuration to mitigate common web vulnerabilities and improve client-side security.

1. Identify which security headers are missing by reviewing the HTTP response headers of the target host (e.g., using curl -I or a browser developer tools).
2. Configure the web server (e.g., Apache, Nginx, IIS) to include headers such as Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options, Content-Security-Policy, X-XSS-Protection, and Referrer-Policy.
3. For each missing header, add the appropriate directive to the server configuration file (e.g., for Nginx add add_header directives in the server block, for Apache use Header always set).
4. Test the changes by sending a request and verifying the headers are present in the response.
5. Deploy the updated configuration to production after testing in a staging environment.

#15 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The web server at 172.19.0.2 is missing common HTTP security headers, such as Content-Security-Policy, X-Frame-Options, or X-Content-Type-Options. In a real-world attack, an adversary could exploit this lack of headers to perform cross-site scripting (XSS) or clickjacking attacks, potentially stealing user credentials or performing unauthorized actions. The business impact includes data breaches, loss of customer trust, and potential regulatory fines, especially if sensitive information is exposed.

Exploitability rationale: Missing HTTP security headers alone are not directly exploitable; they require an additional vulnerability (e.g., XSS) to be present, and the service is internal with no known public exploit.

Remediation: Add missing HTTP security headers to harden the web application against common attacks.

1. Identify which security headers are missing by reviewing the scan report or using a tool like securityheaders.com.
2. Add the Content-Security-Policy header with appropriate directives to mitigate XSS and data injection attacks.
3. Add the X-Content-Type-Options: nosniff header to prevent MIME type sniffing.
4. Add the X-Frame-Options: DENY or SAMEORIGIN header to prevent clickjacking.
5. Add the Strict-Transport-Security header to enforce HTTPS connections.
6. Add the X-XSS-Protection: 1; mode=block header (though deprecated, it may be added for legacy browser support).
7. Deploy the updated configuration to the web server (e.g., Apache, Nginx, IIS, or application code).
8. Verify the headers are correctly served using curl or browser developer tools.

#16 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The web server at 172.19.0.2 is missing common HTTP security headers, such as Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options. This misconfiguration exposes the application to real-world attacks like clickjacking, where an attacker can trick users into performing unintended actions by embedding the site in a malicious iframe, and MIME type sniffing attacks that could lead to cross-site scripting. The absence of these headers also weakens defenses against content injection and data exfiltration. While rated as informational severity, the business impact includes increased risk of client-side attacks, potential regulatory non-compliance (e.g., PCI-DSS, OWASP Top 10), and erosion of user trust if exploited.

Exploitability rationale: The missing security headers are a misconfiguration that increases client-side attack risk but require user interaction and network access to an internal host, with no direct remote exploit or known CVE.

Remediation: Add missing HTTP security headers to harden the web application against common attacks like XSS, clickjacking, and MIME sniffing.

1. Identify the web server software (e.g., Apache, Nginx, IIS) and locate its configuration file.
2. Add the 'Strict-Transport-Security' header with a max-age of at least 31536000 and includeSubDomains if HTTPS is enforced.
3. Add the 'X-Content-Type-Options' header set to 'nosniff' to prevent MIME type sniffing.
4. Add the 'X-Frame-Options' header set to 'DENY' or 'SAMEORIGIN' to prevent clickjacking.
5. Add the 'X-XSS-Protection' header set to '1; mode=block' for legacy browser protection (though modern browsers may ignore it).
6. Add the 'Content-Security-Policy' header with a restrictive policy (e.g., default-src 'self') to mitigate XSS and data injection.
7. Add the 'Referrer-Policy' header set to 'strict-origin-when-cross-origin' to control referrer information.
8. Add the 'Permissions-Policy' header to restrict browser features (e.g., 'camera=(), microphone=()').
9. Restart or reload the web server to apply the changes.
10. Verify the headers are present using a tool like curl or an online security header checker.

#17 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The web server at 172.19.0.2 is missing security headers such as Content-Security-Policy, X-Frame-Options, or Strict-Transport-Security. In a real-world attack, an adversary could exploit the absence of X-Frame-Options to perform clickjacking, tricking users into unintended actions, or leverage missing Content-Security-Policy to inject malicious scripts (XSS). This increases the risk of data theft, session hijacking, or reputational damage, potentially leading to financial loss or regulatory non-compliance.

Exploitability rationale: Missing security headers are not directly exploitable; they require additional vulnerabilities or user interaction to be leveraged.

Remediation: Add missing HTTP security headers to the web server configuration to mitigate common web vulnerabilities such as clickjacking, MIME sniffing, and cross-site scripting.

1. Identify which security headers are missing by reviewing the HTTP response headers of the web application (e.g., using curl -I or browser developer tools).
2. Configure the web server (e.g., Apache, Nginx, IIS) or application framework to include the following recommended headers: Content-Security-Policy, X-Content-Type-Options: nosniff, X-Frame-Options: DENY or SAMEORIGIN, Strict-Transport-Security (if HTTPS), X-XSS-Protection: 0 (or disable), and Referrer-Policy.
3. For each header, set appropriate values based on the application's requirements (e.g., Content-Security-Policy should restrict allowed sources for scripts, styles, etc.).
4. Test the changes by sending requests and verifying the headers are present and correctly configured.
5. Monitor for any breakage in functionality and adjust header values as needed, then deploy to production.

#18 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The web server at 172.19.0.2 is missing security headers such as Content-Security-Policy, X-Frame-Options, or Strict-Transport-Security. In a real-world attack, an adversary could exploit the absence of X-Frame-Options to perform clickjacking, tricking users into unintended actions, or leverage missing CSP to inject malicious scripts (XSS). This increases the risk of data theft, session hijacking, or reputational damage, potentially leading to financial loss and regulatory non-compliance.

Exploitability rationale: Missing security headers are a configuration weakness that requires additional attack vectors like XSS or user interaction, not directly exploitable for code execution or data access, and no CVE or known public exploit exists.

Remediation: Add appropriate security headers to HTTP responses to mitigate common web vulnerabilities.

1. Identify the web server software (e.g., Apache, Nginx, IIS) running on 172.19.0.2.
2. Review the list of missing security headers from the scan output (e.g., Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, Referrer-Policy, Permissions-Policy).
3. For each missing header, determine the appropriate value based on your application's requirements and security best practices.
4. Configure the web server to send the selected headers in all HTTP responses (e.g., for Nginx add add_header directives in the server block; for Apache use Header always set directives; for IIS use HTTP Response Headers in IIS Manager or web.config).
5. Restart or reload the web server to apply the changes.
6. Verify the headers are present using a browser developer tools or a tool like curl -I https://172.19.0.2.

#19 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The web server at 172.19.0.2 is missing security headers such as Content-Security-Policy, X-Frame-Options, or Strict-Transport-Security. An attacker could exploit this by injecting malicious scripts (XSS) or performing clickjacking attacks, potentially stealing user data or executing unauthorized actions. This increases the risk of data breaches and reputational damage, especially if the server handles sensitive information or user sessions.

Exploitability rationale: Missing security headers are a configuration weakness that requires an additional vulnerability (e.g., XSS) to be exploitable, and no public exploit or CVE exists for this finding alone.

Remediation: Add missing HTTP security headers to mitigate common web vulnerabilities such as clickjacking, MIME sniffing, and cross-site scripting.

1. Identify the web server (e.g., Apache, Nginx, IIS) and locate its configuration file.
2. Add the 'X-Content-Type-Options: nosniff' header to prevent MIME type sniffing.
3. Add the 'X-Frame-Options: DENY' or 'SAMEORIGIN' header to prevent clickjacking attacks.
4. Add the 'Content-Security-Policy' header with a restrictive policy (e.g., default-src 'self') to mitigate XSS and data injection.
5. Add the 'Strict-Transport-Security' header (e.g., max-age=31536000; includeSubDomains) if HTTPS is enforced.
6. Add the 'Referrer-Policy' header (e.g., strict-origin-when-cross-origin) to control referrer information.
7. Add the 'Permissions-Policy' header to restrict browser features.
8. Restart or reload the web server to apply changes and verify headers using curl or browser developer tools.

#20 Low HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The web server at 172.19.0.2 is missing critical HTTP security headers such as Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options. In real-world attacks, this omission enables clickjacking, cross-site scripting (XSS) via inline scripts, and MIME-type sniffing, which can lead to session hijacking or data exfiltration. The business impact includes potential data breaches, regulatory non-compliance (e.g., PCI DSS, GDPR), and reputational damage from successful client-side attacks.

Exploitability rationale: Missing security headers do not directly enable exploitation; they increase risk of client-side attacks only if other vulnerabilities exist, and the service is on an internal IP.

Remediation: Add recommended security headers to HTTP responses to mitigate client-side attacks and improve overall security posture.

1. Identify which security headers are missing by reviewing the scan results or using a tool like SecurityHeaders.com.
2. For each missing header, determine the appropriate value based on your web application's requirements and OWASP guidelines (e.g., Content-Security-Policy, X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Strict-Transport-Security: max-age=31536000; includeSubDomains).
3. Implement the headers in your web server configuration (e.g., Apache, Nginx, IIS) or via application code/framework middleware.
4. For Apache, add Header directives in .htaccess or virtual host configuration; for Nginx, use add_header directives in server or location blocks.
5. Test the changes using curl or browser developer tools to ensure headers are present in responses.
6. Repeat the scan to confirm that all missing headers are now set and no new issues are introduced.

#21 Low robots.txt endpoint prober (severity: info) [tags: miscellaneous, misc, generic, discovery]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The vulnerability is the exposure of a robots.txt file on the web server at 172.19.0.2, which is typically used to guide search engine crawlers but can inadvertently reveal hidden or sensitive directories (e.g., admin panels, backup files). In a real-world attack scenario, an attacker probes for robots.txt during reconnaissance to map the web application's non-public paths, potentially leading to discovery of vulnerable endpoints or administrative interfaces. The business impact includes increased risk of targeted attacks against exposed areas, such as unauthorized access to confidential data or exploitation of unpatched systems, especially if the revealed paths are not properly secured.

Exploitability rationale: The finding is informational, revealing potential hidden paths via robots.txt, but requires further exploitation steps and is on an internal host with no known exploits or CVEs.

Remediation: The robots.txt file is a publicly accessible resource intended to guide crawlers, but exposing sensitive paths can aid attackers in reconnaissance; review and sanitize its contents.

1. Review the robots.txt file at http://172.19.0.2/robots.txt and identify any listed directories or files that are not intended for public access (e.g., admin panels, private APIs).
2. Remove any disallowed or allowed paths that reveal sensitive internal resources, moving those paths to a separate, non-public configuration if they must be blocked for crawlers.
3. Ensure no confidential information (e.g., backup files, configuration details) is inadvertently exposed, and consider using HTTP authentication or IP whitelisting for any sensitive directories instead of relying solely on robots.txt.

#22 Low robots.txt file (severity: info) [tags: miscellaneous, misc, generic, vuln]

Host: 172.19.0.2

Risk score: 0.38 **(asset criticality 0.5)**

Context: The robots.txt file on 172.19.0.2 is a publicly accessible text file that instructs web crawlers which parts of the site to avoid. While not a vulnerability itself, it can inadvertently expose sensitive directories or files (e.g., admin panels, backup files) to attackers, who may use this information to target restricted areas. In a real-world scenario, an attacker could enumerate the listed paths to discover hidden resources, potentially leading to unauthorized access or data breaches. The business impact includes increased risk of information disclosure and targeted attacks, which could compromise confidentiality and integrity of sensitive data.

Exploitability rationale: The robots.txt file is an informational finding that may reveal directory paths, but without evidence of sensitive content and given internal network exposure, the exploitability is low.

Remediation: A robots.txt file is intended for search engine crawlers, but exposing directory paths can aid reconnaissance; restrict or remove sensitive entries if they reveal internal paths.

1. Review the robots.txt file to identify any disallowed paths that expose sensitive or internal directories (e.g., /admin, /config).
2. Remove or obfuscate any entries in robots.txt that reveal internal paths not intended for public discovery.
3. If the robots.txt file is not required for legitimate search engine indexing, consider removing it entirely or restricting access to it via web server configuration (e.g., block with .htaccess or Nginx rule).

Ranked Summary

Rank	Score	Exploitability	Finding	Host
1	0.665	High	Sensitive Configuration Files Listing - Detect (severity: medium) [tags: config, listing, exposure, edb, vuln]	172.19.0.2
2	0.65	High	DVWA Default Login (severity: critical) [tags: dvwa, default-login, vuln]	172.19.0.2
3	0.5	Medium	Cookies without Secure attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]	172.19.0.2
4	0.5	Medium	Cookies without HttpOnly attribute - Detect (severity: info) [tags: misconfig, http, cookie, generic, vuln]	172.19.0.2
5	0.5	Medium	FingerprintHub Technology Fingerprint (severity: info) [tags: tech, discovery]	172.19.0.2
6	0.5	Medium		172.19.0.2

Rank	Score	Exploitability	Finding	Host
			Missing Cookie SameSite Strict (severity: info) [tags: misconfig, samesite, cookie, vuln]	
7	0.5	Medium	README.md file disclosure (severity: info) [tags: exposure, markdown, files, vuln]	172.19.0.2
8	0.5	Medium	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
9	0.5	Medium	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
10	0.38	Low	WAF Detection (severity: info) [tags: waf, tech, misc, discovery]	172.19.0.2
11	0.38	Low	Gitignore Config - Detect (severity: info) [tags: exposure, tenable, config, git, vuln]	172.19.0.2
12	0.38	Low	Wappalyzer Technology Detection (severity: info) [tags: tech, discovery]	172.19.0.2
13	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
14	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
15	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
16	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
17	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
18	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
19	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
20	0.38	Low	HTTP Missing Security Headers (severity: info) [tags: misconfig, headers, generic, vuln]	172.19.0.2
21	0.38	Low	robots.txt endpoint prober (severity: info) [tags: miscellaneous, misc, generic, discovery]	172.19.0.2
22	0.38	Low	robots.txt file (severity: info) [tags: miscellaneous, misc, generic, vuln]	172.19.0.2

Risk score = (CVSS/10 × 0.5) + (Exploitability × 0.3) + (Asset × 0.2). Exploitability: High=1.0, Medium=0.5, Low=0.1.